CMPUT274—Assignment 3 (Fall 2026)

R. Hackman

Due Date: Friday November 6th, 8:00PM

Per course policy you are allowed to engage in *reasonable* collaboration with your classmates. You must include in the comments of your assignment solutions a list of any students you collaborated with on that particular question.

For each assignment you may not use any Python features or functions not discussed in class! This means, for example, you may not use loops, casting, any built-in functions or methods we have not discussed. If you are in doubt, you may ask on the discussion forum — however remember if you are including your code, or even your plan to solve a question, you must make a private post. Use of disallowed features will result in a grade of zero on the question(s) in which they were used.

All functions you write must have a complete function specification as presented in the course notes. Failure to provide function specifications for functions will lead to the loss of marks.

Unless explicitly forbidden by a question, you are always allowed to write additional helper functions that may help you solve a problem. In fact in some cases it will be *necessary* to write helper functions!

Allowed Keywords and Functions

· `True` · `False` · `def` · `and` · `assert` · `if` · `elif` · `else` · `import` · `from` · `not` · `or` · `return` · `testExact` · `testWithin` · Arithmetic operators · String/Tuple subscription, slicing, and concatenation · `foldr` · `foldl` · `map` · `filter` · `buildList` · `lambda` · `LL` · `cons` · `first` · `rest` · `isEmpty` · `len`

1. **Functions Producing Functions** In this question you are asked to write several small functions each of which produce a function as their result.

   - `reverseParams` is a function which takes a single parameter that is a binary function `f` of the form `(X Y -> Z)`. Your function should produce the function of the form `(Y X -> Z)` that has the exact same behaviour as `f` except the order of the parameters is swapped.
     - Example: `reverseParams(lambda x, y : x / y)(2, 1) -> 0.5`
   - `conjunction` is a function which takes a single parameter that is a `LList` of predicate functions of the form `(X -> bool)` and returns a function of the form `(X -> bool)` that is the conjunction of all the predicates in the given `LList` applied to the given argument.
     - Example:

```
isEven = lambda x: x % 2 == 0
isNonNeg = lambda x: x >= 0
isSmall = lambda x: x < 100
allAbove = conjunction(LL(isEven, isNonNeg, isSmall))
filter(allAbove, LL(-4, 0, 3, 6, 98, 99, 102, 103)) -> LL(0, 6, 98)
```

   - `repeat` is a function which takes two parameters the first being a unary function `f` of the form `(X -> X)`, and the second being a `Nat n`. Your function should produce a function of the type `(X -> X)` which performs the application of `f n` times to the provided argument.
     - Example: `repeat(lambda x: x*2-1, 3)(3)->17`
   - `until` is a function which takes two parameters:
     - `p` a predicate function of the form `(X -> bool)`
     - `f` a unary function of the form `(X -> X)`

     Your function should return a unary function of the form `(X -> X)` which is the function that repeatedly applies `f` to its argument as many times as is necessary until the result meets the provided predicate.
     - Example: `until(lambda x: x > 100, lambda x: x*2)(3) -> 192`

   Deliverables: For this question include in your final submission zip the python file containing your solutions named `FUNctions.py`

2. **Sorting with a Spatula** For this question you will be implementing a function `spatulaSort` which takes a single `LList` parameter and returns a copy of that `LList` sorted in non-decreasing order. However, unlike the sorting algorithms we have discussed before you have an additional restriction.

   In the provided file for this question you will find a function `flip` which takes two parameters first a `LList` and second a `Nat k`. The function flip returns a copy of the given `LList` where the first `k` elements have been reversed (or flipped, as if the elements are stacked on top of each other and a spatula was inserted at the kth element and flipped that section of the stack). For example, `flip(LL('a','b','c','d','e','f'), 3) -> LL('c', 'b', 'a', 'd', 'e', 'f')`

   In writing your solution for this question you may not use `cons` at all, and instead may only use `flip` (which does use `cons`, the only permitted use) to produce new `LLists`.

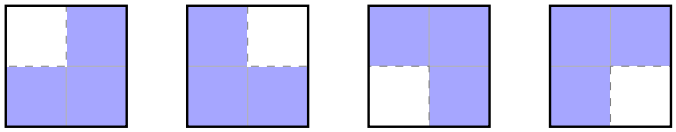
As always, you may write as many helper functions as you see fit. However, none of them may use `cons` as none of your code may use `cons`, only the provided function `flip` may do so.

Note: this sorting problem is a well-known one by another name, it is purposely presented to you without its proper name so as not to incentivize googling the solution.

Deliverables: For this question include in your final submission zip the python file containing your solution named `spatula.py`

3. **Tiling a deficient board**

A *deficient board* is a square grid of side 2^k (for some $k \geq 0$) with exactly one square removed, called the *hole*. An *L-tromino* is an L-shaped tile that covers exactly three squares of a 2×2 block. There are four orientations, shown below:



Your goal is to write a function to help find a *tiling* that covers every square of the board *except a given hole* with L-trominoes so that no two trominoes overlap and no tile extends past the edge of the board.

A board of side 2^k has $4^k - 1$ squares to cover, and $4^k - 1$ is always divisible by 3, so $(4^k - 1)/3$ trominoes are needed. Such a tiling *always exists*. In order to find such a tiling we will use a *divide-and-conquer* strategy described later. First, we will define our data representations of our deficient board and our trominoes.

- A `Square` is a 2-tuple `(row, col)`, where both `row` and `col` are `Nats`. Rows are numbered from the top starting at 0, and columns from the left starting at 0.
- A `Region` of the board is a 2-tuple (Square, Nat). The first item is the `Region's` top-left square and the second its side length, which is always a power of 2.
- A `tromino` is a 3-tuple of unique squares, in which each square is adjacent to at least one other square and represents an L-shape.
- A **tiling** is a `LList` of trominoes.

The Divide and Conquer Strategy

For a deficient board of side length 2^0, the only square is the hole, so no trominoes are needed. Similarly, for a deficient board of side length 2^1 there is trivially only one tromino that fits in the tiling. For a larger board with side $s = 2^k$:

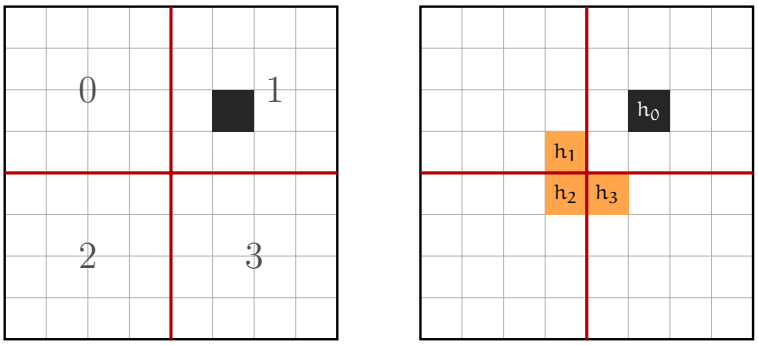
(a) **Divide.** Split the board into four quadrants, each of side $s/2$, numbered 0 (top-left), 1 (top-right), 2 (bottom-left), 3 (bottom-right). The hole lies in exactly one quadrant, the other three quadrants do not have holes in them.

(b) **Modify.** Place a single tromino at the *center* of the board, covering the one center square belonging to each of the three quadrants that do *not* contain the hole.

(c) **Conquer.** Every quadrant is now a smaller deficient board: the quadrant with the original hole has that hole, and each of the other three has a "hole" where the central tromino covers it. Solve the four quadrants recursively.

(d) **Combine.** The tiling of the whole board is the central tromino together with the four recursive tilings.

The image below shows how to "divide" an 8×8 board with the hole at `(2, 5)`, which lies in quadrant 1.

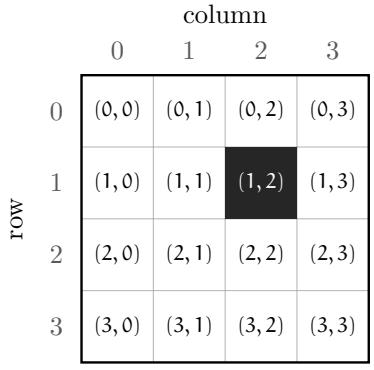


(a) Split into quadrants

(b) Central tromino placed forming three more "holes"

You must write the function `tileDeficientBoard` which takes two parameters, the first a `Region` which represents a board (that does not necessarily have to begin at (0,0)) and a `Square` which is the coordinate of the hole in the given board. Your function should return the `Tiling` of the given deficient board produced by following the process described above. Note that for sufficiently sized boards there are multiple valid tilings, following the divide-and-conquer strategy described above should lead to a distinct tiling each time however.

For example the call `tileDeficientBoard(((0,0) 4), (1,2))` represents a request to tile the 4×4 deficient board with a hole at square $(1, 2)$, this is illustrated below:



Deliverables: For this question include in your final submission zip the python file containing your solution named `tiling.py`